

Lec 29 Practical Exercise - 2

Lec 29 Practical Exercise - 2

Lecture	29 / 63 + intro
Duration	26:31
Captions	English (auto-generated)
Words	3,821
Video	f9FV2DbbxnA
Playlist	Fundamentals of Generative AI and LLMs

Auto-generated YouTube captions, lightly cleaned (removed [music] tags, grouped into paragraphs). These are **verbatim lecture captions**, not edited notes.

Transcript

[00:00] Hello everyone. Welcome to the hands-on session on fundamentals of generative AI and large language models theory and practice. In today's session, we'll be learning the implementation of conditional variational autoencoder, which is an improvisation over variational autoencoder which we have seen in the last hands-on session. So, we have seen in case of VAE, that is variational autoencoder, which makes use of the latent vector that is Z , which is dependent on the probability distribution of input data, which makes use of mainly the mean value represented as μ and the variance of the data or maybe the standard deviation of the data, as well as it is also dependent on the epsilon value, which is a small

[00:48] random noise which is generated. So, with the help of such latent space, uh compared to a traditional autoencoder, we have seen that the latent space was very smooth and the transition or the interpolation between the samples was very smooth and efficient. But, one of the drawback what has happened in case of variational autoencoder is that it the reconstruction of the image does not make use of any class. It is not dependent on any target variable or the output label. Due to which the reconstruction of the image is very poor.

[01:22] So, in order to improvise this reconstruction in case of variational autoencoder, we have this conditional variational autoencoder, which acts upon the input data, which creates a latent vector which is once again dependent on the μ value as well as the standard deviation and a small random noise. And when this uh when this latent vector is given to the decoder, it performs the reconstruction of the image, but it also depends on the output variable. So, it takes into account both the input image as well as its output label. So, it is basically a posterior probability or we can say that it is dependent on the conditional probability. So, let us see its implementation in today's hands-on.

[02:13] So, in order to have this conditional variational autoencoder, we initially import the necessary libraries. So, the libraries required are TensorFlow, which is a deep learning framework required for building the neural networks. From Keras, we import the layers in order to have a combination of input layers, hidden layers, and output layers. Also, we import the model in order to build the conditional variational autoencoder. And we import NumPy as np because we need to have certain arrays in this case. We do a lot of computation or we perform one-hot encoding on the output data. So, that is where we deal with integers or numbers, and that is why we need this numerical Python.

[02:58] And also for data visualization purpose, we have this matplotlib.pyplot, which is in short called as plt. Now, coming to loading of the data set. So, we have used the same MNIST data set in case of variational autoencoder. So, to check or to give a comparison between VAE and conditional VAE, we are making use of the same MNIST data set, which is once again a handwritten digit data set comprising of 10 classes, wherein the digits vary from 0 to 9. So, let us do some kind of preprocessing on this data set once we load the data set. So, from Keras datasets, we load this MNIST data set and put it into four different variables, namely X_train, Y_train, which comprises the training set, X_test, and Y-test, which comprises the

[03:54] test set. So, as mentioned earlier, the difference the only difference between conditional VAE and VAE is that in case of VAE, we were ignoring the output label, but in this case of conditional VAE, we are making use of Y, that is the output label. That's why the output label is now divided into training set and the test set. That's why we have two variables, Y-train and Y-test. So, now converting this data type as floating data type and then performing the normalization. So, in the previous hands-on, we have seen why to normalize the data in the range of 0 to 1 because we have the grayscale image, which has 256 intensity values varying from 0 to 255. Since the maximum value is 255, we divide each of the values by 255. So,

[04:45] that process is done here. And then we reshape the training as well as the test data, which is the input data, such that it has 784 different pixel values. Since the input image is of size 28 pixels in width and 28 pixels in height, we put together a convert this 2D matrix 28 cross 28 into variables or input variables or also called as

input features, which are 784 in number. So, since we want to ignore Y in our case of X-train and X-test, we have this number minus one, so that we ignore the last column.

[05:26] Now, coming to the important task here of one-hot encoding, which is explicitly present only in the conditional VAE. So, we perform one-hot encoding on the Y-train as well as Y-test. So, what is this one-hot encoding? Let us see. » As mentioned earlier, MNIST dataset has 10 different classes. And the digits vary from 0 to 9. So, instead of having this integer representation given to each of these classes, what can be done is, in case of one-hot encoding, the presence of any digit is indicated with the help of ones and absence in terms of zeros, just like a binary format of data. Presence of the data indicated using ones and absence is shown using zero. So, if I have to give a representation for zero, then this

[06:34] would be 1 0 0 0 0. So, the vector length should be 10 because we have 10 different classes. And the first place indicates this index zero, nothing but the digit zero. So, now coming to class one or digit one, that is represented using a one in the first index. So, rest all of the places, nine different places, become zero. So, the vector length should be 10. Should be equal to 10, which is same as the number of classes.

[07:21] Should be same as the number of classes. Similarly, to represent two, we have 0 0 1 0 0. Rest all are zeros. So, this is how we have representations for 10 different digits. 6 7 We have this as one. So, the representation of nine wherein the last digit becomes one. So, this is one hot encoding. So, in order to perform one hot encoding, we take this TensorFlow Keras from the utilities. We convert this Y train and Y test into categorical data.

[08:09] That is Y train having 10 classes. Similarly, Y test having 10 classes. So, when we try to display this Y train and Y test, we get a vector format of data comprising of length 10 and wherever digit is present, that is indicated with the help of one. Rest all places would become zero. So, this is the initial pre-processing of the data set. Now, coming to defining the sampling layer as as we have done in case of VAE, it remains the same except that we also include the output variables. That's why we need to define the number of classes.

[08:44] So, in case of VAE, we have defined only the latent dimension which was 16 input variables. Apart from this parameter, in the case of conditional VAE, we include the number of classes as 10. So, this is the only inclusion or extra what we have added in case of conditional VAE. The rest of the code remains the same. That is we create a sampling layer which is which is determined using the inputs. The inputs which comprises of the mean as well as the variance. So, we get this vector or the latent vector Z.

[09:24] We get the latent vector Z which is a combination of mu plus sigma into epsilon. Mu refers to the mean. Sigma refers to the standard deviation and epsilon

refers to the random noise. Usually a Gaussian noise. So, we have already seen how to represent this μ how to represent the standard deviation from the variance in the last hands-on. So, what we do is we get the epsilon from this random function which is present in the TensorFlow which is a normally distributed data and the shape of this noise is dependent on the shape of the mean. So, shape will be equal to 16 because it's a combination of 16 different latent variables. So, finally we get this Z which is a latent vector which is sum of

[10:24] mean plus exponential of 0.5 into the variance into the epsilon. So, in the last hands-on we have already seen how exactly we get this exponential term. Now, defining the encoder part. So, encoder it gets the input from directly from the input images. So, input images 784 input variables we have so shape will be equal to 784 which acts as the input and that is defined in a variable called as `image {underscore} input`. Now, apart from this image input we should also define a new variable called as `label {underscore} input`. So, what is our `label {underscore} input`? This will be equal to from layers we take the input and shape of that input as nothing but the number of classes present in the input data. So, in our case the number

[11:14] of classes will be equal to 10 which is already defined earlier. So, now the encoder input is the concatenation of image input followed by the label input. So, side by side or one after the other we create this encoder input. So, the encoder input looks in this way. We'll have X followed by the classes. Unlike the previous case, we were interested only in X. That's why we would always consider X train and X test. From the data we would from the data which was a tuple, we would consider only the zeroth index of the data. But in this case, we need both X as well as Y. And Y refers to the number of classes.

[12:06] So, once we define this encoder input, now coming to considering a fully connected layer with the 256 different neurons, let the activation function be ReLU. We are always free to use different activation function. Mainly we use a ReLU because it tries to add maximum non-linearity to the data because it has a fundamental equation or a mathematical representation as maximum of 0, the input. So, whenever the input is negative, it will give the value as 0.

[12:36] And whenever the input is positive, the same value will be retained. So, that is one of the best advantage what ReLU has. That's why most of our hands-on we are trying to use the activation function as ReLU. So, we have defined this fully connected layer, after which we consider one more fully connected layer, two more fully connected layer, one for the μ , other one for getting the variance or the standard deviation. So, once again this mean or the standard deviation is based on the latent dimensions what we have defined. So, what is the dimension we have mentioned is 16. So, based on that dimension, we get the mean. So, 16 different mean values, 16 different variance values.

[13:20] So, on this we create the sampling layer. We do a random sampling on the mean as well as the variance. So, now what does our encoder consist of? It is a model which comprises the image input as well as input label, which is nothing but the Y value, as well as the latent vector Z, which is dependent on mean and the variance. So, let us give the name to this architecture as encoder. So, already we have defined the encoder and the sampling layer. Now, coming to the decoder.

[13:53] So, even in the decoder, one extra thing what we do is we are considering the number of classes, that is the target variable. So, input is dependent on shape of the latent dimension, which is 16, after which we take the label decoder. So, label decoder dependent on or the shape of the label decoder is dependent on the number of classes present, which is 10 in number here. So, totally, what is the input that is going to the decoder? It is concatenation of whatever input that is Z, followed by the number of classes. So, what is the decoder input?

[14:35] will be Z and the number of classes. So, classes is nothing but Y. The output Y. So, now we define a fully connected layer, which is a dense layer comprising of 256 neurons, activation function ReLU adding non-linearity, and the final dense layer, which is a fully connected layer with 784 different variables, because we have to construct or reconstruct back the given image. That's why we have to get the same shape as that of the original image. That's why the number of neurons what we are taking is 784. And the activation function is sigmoid, because we have already normalized the given input in the range 0,1.

[15:21] And we have seen many of instances that sigmoid will always give the values, the probability value lying between zero to one. So, that is the reason why we are using the activation function as sigmoid in this case. So, now what does the decoder architecture consist of? Decoder is a model which is dependent on the latent dimension or the latent input which is Z, the label decoder which is Y, the decoder output which is already defined here, that is we need the 784 variables back or 784 pixel values back.

[15:59] And let the name given to this decoder to this decoder architect architecture is decoder. So, this is all about three different layers, the encoding layer, sampling layer, and then the decoding layer. Now, coming to the actual conditional VAE model. So, we construct this conditional VAE model from flow or the Keras model class. And then we do the initial initialization of the encoder, the decoder, and whatever inputs that have to go to the encoder as well as the decoder. So, the inputs are image input and the label, which are the inputs, nothing but X, out of which we derive the Z. Z is a latent vector. We try to have this latent vector which is little flexible when compared to the fixed vector or a

[16:51] fixed point in case of a auto encoder. So, Z dependent on me mean and the variance, and return the decoder, the decoder which is acting upon the latent vector

as well as the output label. So, now coming to how to how do we train this CVAE or a conditional VAE? So, what we do is we take the image, we take the label, and we take the data. So, data of zero, whatever the first index present, that is the image input, we consider that and perform the gradient or we need to calculate the gradients. So, what are these gradients or how is it important?

[17:37] So, gradients will be calculated in the back propagation, which is which is derivative of loss divided by the derivative of weights. So, let us see how how is the new weight calculated. New weight will be equal to old weight minus the learning rate into the gradients. Now, this gradient is calculated during the back propagation. And the loss or a ΔL what we consider is based on the reconstruction loss as well as the KL loss. Just like the normal VAE what we have seen in the last demonstration. So, what is this reconstruction loss? Reconstruction loss is nothing but the binary cross entropy loss that we consider. So, which will be equal to $-\sum y \log y - \sum (1-y) \log (1-y)$.

[18:49] So, y is the actual output and $\hat{y}$ is the reconstructed or the predicted output. And what is the scale divergence? KL divergence will be $-\frac{1}{2} \sum \left(1 + \log \frac{\sigma^2}{\mu^2} - \frac{\mu^2}{\sigma^2} \right)$. This is the KL divergence loss. So, there is no difference between the BCE and KL what we consider for VAE. But, the only thing what we need to include here is the output. So, we calculate the reconstruction loss, but we also calculate under reconstruction loss, we take the mean of those losses. So, which which is already explained in the previous hands-on or let me summarize it here. So, we have BCE loss or average BCE loss will be BCE loss one plus BCE loss two.

[19:52] Likewise, we have 16 different loss with this present for 16 different latent variables. So, when we take the average, that is the mean loss what we get here. That's why we have this tensor flow reduce mean value. So, we are calculating the binary cross entropy and then taking the mean which gives us the reconstruction loss. So, this reconstruction loss is based on the actual output that is image and the reconstructed image that is $\hat{Y}$. So, this is Y , Y is image and $\hat{Y}$ is this reconstruction. Same thing to do with the KL loss. It is $-\frac{1}{2} \sum \left(1 + \log \frac{\sigma^2}{\mu^2} - \frac{\mu^2}{\sigma^2} \right)$. So, this is already derived in the previous hands-on. So, total loss will be equal to

[20:45] reconstruction loss plus the KL loss. So, based on this loss, we calculate the gradients or we have to see how much weights we need to adjust during the back propagation in order to reduce the loss, in order to minimize the loss. So, that is a main aim that that is involved in the back propagation concept. So, in order to have that, we can have different optimizers. So, different optimizer have their own weight updation equations. Maybe some optimizers are dependent on momentum, some don't have momentum, some may be dependent on the mean value, some may be dependent on the velocity, and so on. So, based on the requirement, we may choose

the optimizers.

[21:29] So, finally, we return the total loss during our epoch, during the various iterations. We try to have the loss values, reconstruction loss as well as the KL loss. So, now coming to creation of conditional VAE. So, conditional VAE comprises of encoder, decoder. We configure it based on some optimizer. So, for simplicity, we have taken Adam because Adam always makes use of the mean estimate and the variance estimate. So, which is the major contribution we have in case of VAE.

[22:03] Now, coming to fitting this on the training data. So, training data comprises of X_train as well as Y_train. So, if we remember the previous hands-on, we have excluded this Y_train. But, this Y_train has to be included in case of conditional VAE. And rest of the things remain the same. The number of epochs could be 10, the batch size could be 128 or 256 based on how many input images we need to convert into batches. So, less the batch, what happens is the number of weight updation will be more. So, in that case, more number of weights we update, less loss we get. But, there is always way to find out the better batch size so that the weight updation will be very much optimized.

[22:49] So, now let us look into the various loss values at different epochs. We can see a combination of KL loss and reconstruction loss, which has given rise to the total loss. So, at the end of the 10th epoch, we see that reconstruction loss, which is 0.21, plus the KL loss, which is $1.05 * 10^{-5}$, as given has given rise to the loss, which is 0.2160. So, if you remember the previous hands-on, the loss in case of VAE was slightly more when compared to this.

[23:26] So, we have we can compare the conditional VAE to the normal VAE and say that the loss is very much minimum here because we are making uh the class separation very much specific here. Whereas, the reconstruction what was happening in case of VAE was not class specific, and it was a more generalized way. So, now coming to the last part of this code, reconstruction of the test image. So, let us uh have the random latent vector. So, how do we have this a latent vector? So, based on the random sampling, so that we have a normal distribution of data, and size of such a data is equal to one and the latent dimension.

[24:10] That means we want to generate one digit, and the digit is based on the number of latent dimensions we have. So, it is 1 {comma} 16. So, let us make it class specific so that we have a proper or a optimized class separation. That is the reason we give some label here. So, let us uh have an example of generating label seven or digit seven. So, label will be equal to TensorFlow from TensorFlow Keras utilities. We convert this into categorical, that is one-hot encoding, which is already discussed in the earlier part of the code. So, we convert this into one-hot encoder uh one-hot vector, and the number of classes are 10.

[24:54] Now, what is this generated image? It is based on whatever prediction that

is done by the decoder. So, decoder makes use of the latent vector that is Z as well as the label. So, let us show the generated image and let it be having the original shape just like the original image shape 28 cross 28 and the color map let it be gray. And let us show the image without showing any x-axis or y-axis. That is why it is turned off here.

[25:25] So, we see that the image is seven is being generated here. Unlike the variational autoencoder, the reconstruction was not class specific. Randomly, we could generate some 10 sample images or 20 sample images without making it class specific. So, the advantage in case of conditional VAE is it can have a better class separation because we once again consider the latent vector which is not a fixed to point which is always flexible, which is dependent on mean as well as the variance value. But here the interpolation between the samples is very much free or very much efficient when compared to the normal VAE or just a basic autoencoder.

[26:12] So, in today's hands-on, we have seen how a conditional VAE has been able to reconstruct the image or generate a given image which is very much class specific. Thank you.